

An Industrial-Scale Sequential Recommender for LinkedIn Feed Ranking

Lars Hertel*, Gaurav Srivastava*, Syed Ali Naqvi, Satyam Kumar, Yue Zhang, Borja Ocejo†, Benjamin Zelditch, Adrian Englhardt, Hailing Cheng, Andy Hu, Antonio Alonso, Daming Li, Siddharth Dangi, Chen Zhu, Mingzhou Zhou, Wanning Li, Tao Huang, Fedor Borisjuk, Ganesh Parameswaran, Birjodh Tiwana, Sriram Sankar, Qing Lan, Julie Choi, Souvik Ghosh†

LinkedIn Inc., USA

lhertel@linkedin.com, gsrivastava@linkedin.com

Abstract

LinkedIn Feed enables professionals worldwide to discover relevant content, build connections, and share knowledge at scale. We present *Feed Sequential Recommender (Feed SR)*, a transformer-based sequential ranking model for LinkedIn Feed that replaces a DCNv2-based ranker and meets strict production constraints. We detail the modeling choices, training techniques, and serving optimizations that enable deployment at LinkedIn scale. *Feed SR* is currently the primary member experience on LinkedIn’s Feed and shows significant improvements in member engagement (+2.10% time spent) in online A/B tests compared to the existing production model. We also describe our deployment experience with alternative sequential and LLM-based ranking architectures and why *Feed SR* provided the best combination of online metrics and production efficiency.

Keywords

Sequential Recommendation, Generative Recommendation, Recommender systems, Ranking

ACM Reference Format:

Lars Hertel*, Gaurav Srivastava*, Syed Ali Naqvi, Satyam Kumar, Yue Zhang, Borja Ocejo†, Benjamin Zelditch, Adrian Englhardt, Hailing Cheng, Andy Hu, Antonio Alonso, Daming Li, Siddharth Dangi, Chen Zhu, Mingzhou Zhou, Wanning Li, Tao Huang, Fedor Borisjuk, Ganesh Parameswaran, Birjodh Tiwana, Sriram Sankar, Qing Lan, Julie Choi, Souvik Ghosh†. 2018. An Industrial-Scale Sequential Recommender for LinkedIn Feed Ranking. In *Proceedings of (Conference acronym ’XX)*. ACM, New York, NY, USA, 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

LinkedIn Feed enables professionals to share knowledge and ideas through text, images, and video. Members viewing the Feed are presented with posts from their network, as well as relevant posts

*Both authors contributed equally to this research.

†Work done while at LinkedIn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym ’XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

from members they are not connected to. In addition, the Feed includes other content such as job changes, job opportunities, and suggested connections. Posts in the Feed are ranked by predicted relevance, defined as the likelihood that a member will take actions such as clicking, liking, commenting, or sharing. Improving the accuracy of these predictions leads to more relevant content and is a key driver of growth for both the LinkedIn Feed and the broader LinkedIn ecosystem. Sequential recommendation methods have shown promise in improving prediction accuracy of member actions. However, there are several challenges in applying sequential recommendation to the LinkedIn Feed:

- LinkedIn has more than 1.2 billion members, and the activity distribution is long-tailed, with some members having an extremely high number of interactions compared to the average.
- The LinkedIn Feed spans a corpus of billions of posts that is both large in scale and highly dynamic. Although posts can persist for weeks, they can receive a large portion of their interactions within the first 24 hours after posting.
- To offer the best experience for all members, the Feed needs to be ranked in real-time as the member visits LinkedIn.

We introduce *Feed SR*, LinkedIn Feed’s sequential recommendation model, and outline how we address these challenges, including:

- Improving relevance for members with long and short interaction histories through our sequential model architecture (4.1) combined with member profile embeddings (4.5).
- Capturing evolving member interests through RoPE [21] (4.4), incremental training (4.6.2), and a recency weighted loss (4.6.3).
- Modeling a large dynamic corpus of items by supplementing the sequence model with late-fused numeric features. (4.2.3, 4.2.4, 4.3)
- CPU and GPU optimizations that allow us to inference *Feed SR* in real-time. (6.2)

We also discuss lessons from our experiments with other approaches like fine-tuning LLMs and alternate architectures like TransAct [27] in Section 5.

2 Related Work

Our work intersects three active research directions: industrial recommendation models with rich feature sets, sequential recommendation with long user histories, and generative recommendation models that amortize computation across multiple candidate scores.

Industrial Recommendation Models. Deep Learning Recommendation Models (DLRMs) combine dense numerical features, sparse categorical IDs, and cross features to deliver high-quality ranking in production systems [7, 17]. Enhancements such as explicit cross layers (e.g., DCNv2) and multi-task sharing (e.g., MMoE) provide practical performance gains in large-scale ranking [15, 25]. These models remain dominant in online settings due to their ability to integrate rich feature interactions.

Sequential Recommendation. Modeling user behavior sequences is essential for capturing temporal patterns and long-range interests. Transformer-based sequence encoders like SASRec and bidirectional masked pretraining such as BERT4Rec improve next-item prediction by attending over long history contexts [14, 22]. Industrial variants extend these ideas with target-aware attention and dynamic retrieval of relevant behaviors [18].

Generative Recommendation Models. Generative recommendation models (GRMs) treat user histories as token sequences and use transformer transduction for next-item prediction, enabling amortized scoring across many targets [30]. HSTU-style architectures have demonstrated competitive quality with improved efficiency for long histories, but their integration with production-level feature stacks is non-trivial. Recent industrial GRM frameworks such as MTGR emphasize retaining engineered features and optimizing system deployment for ranking at scale [11]. OneRec similarly explores iterative preference alignment in a unified generative retrieve-and-rank framework [9].

Efficient Training and Inference. Efficient attention kernels such as FlashAttention reduce memory movement and latency for long sequences, which is crucial for transformers in production recommender systems [8]. Production-ready libraries like TorchRec further support scalable recommendation training and inference [13].

Our work extends these lines by integrating a sequential model with rich production feature sets under strict latency and throughput constraints in LinkedIn Feed ranking, while retaining high offline and online quality.

3 Background

3.1 LinkedIn Feed

LinkedIn's mission is to connect the world's professionals to make them more productive and successful. As part of this mission, LinkedIn Feed serves to provide access to ideas, learning, and inspiration from members' networks and beyond. To achieve this, the Feed system optimizes for a variety of actions, among them:

- (1) Long Dwell: dwelling on a post longer than a specified threshold of time, depending on the post-type
- (2) Contribution: a like, comment, or share on a post.

To generate a list of posts for a member, candidate posts are first retrieved from multiple sources [10, 19] and combined. Then a ranking model predicts the likelihood of each action. The resulting scores are combined via weights in an objective function that is used to rank the posts. Finally, additional business logic is applied on the ranked post list.

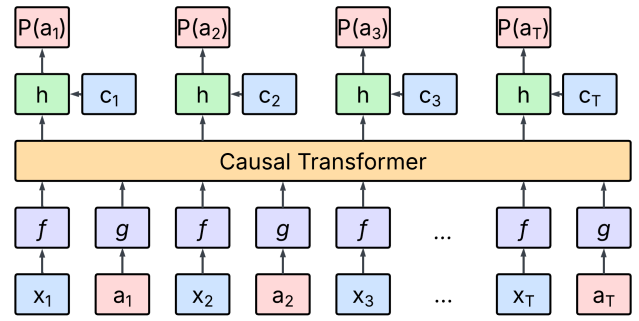


Figure 1: The *Feed SR* model architecture.

3.2 Feed ranking model

The existing Feed ranking model uses post impressions as training samples. The labels are derived from the possible actions. Impressions that do not have a click are retained randomly with 0.1 probability. The model is initially trained on three weeks of Feed data, and then incrementally trained on a daily basis. The ranking model uses a large number of features that can be divided into

- Numeric features: these are dominated by post interaction counts and affinity counts. The latter refers to the number of times a member interacted with another member, an object type, or other dimension.
- Content embeddings: text embeddings externally computed with a text embedding model and statically served to the model [20].
- ID embeddings: high cardinality embeddings learned as part of the model training, for example, for Feed actors.
- Categorical features: examples are post type, device type, or retrieval source.

The model is implemented in Tensorflow and uses two separate neural network towers. The first tower is for passive tasks such as click, skip, and long-dwell. The second tower predicts active actions such as like, comment, and share. Each tower uses a DCNv2 module, followed by multiple fully-connected dense-gating layers [3], and a multi-task output.

The model is trained using a binary cross-entropy loss. A weight of 10 is applied to negative samples based on the previously mentioned down-sampling. Models are evaluated offline using AUC.

Online, the ranking model is scored on CPU in a Java stack. One of the key online metrics is *time spent*.

4 Feed SR – Experiments and offline results

4.1 Architecture overview

Feed SR is a new ranking system for the LinkedIn Feed based on sequential recommendation. The model architecture is based on the ranking model in [30] and enhanced to fit LinkedIn Feed's specific use case, product requirements, and infrastructure. Figure 1 shows a diagram of the model. As shown in Figure 1, we interleave posts with actions similar to [30]. The sequence of interleaved posts and actions are processed by a number of transformer blocks as described in Section 4.2.2 with a causal attention mask [24].

After the transformer blocks, outputs corresponding to interleaved action inputs are discarded as shown in Figure 1 [30]. The result is concatenated with additional context features (Section 4.2.3) and further processed by a "head-architecture" deep neural network with a multi-task output (Section 4.2.4). The multi-task output predicts the actions that the member took on the post. These are the same actions as for the existing architecture in Section 3.2. The model is trained on member interaction histories to predict actions at all points in the interaction sequence. The loss function is a binary cross-entropy loss with weighting as described in Section 4.6.3 and is applied to all sequence positions. During inference, the candidates to be ranked are appended to the end of the sequence and scored at once. More details on inference can be found in Section 6.2.2.

4.2 Architecture Details

4.2.1 Model Input. The model input consists of a history of impressed Feed posts interleaved with the actions taken by the member on each post. Histories are formed from one year of training data (defined in Section 3.2). For each member, we keep the most recent $T=1000$ impressions, ordered chronologically from oldest to newest. A post at sequence position t is represented by J features $\{x_{t,1}, \dots, x_{t,J}\}$. Each feature gets encoded by a feature-specific transform f_j such as embedding lookup, log transform, or identity. The resulting post representation is

$$X_t = \text{concat}(f_1(x_{t,1}), \dots, f_J(x_{t,J})) \in \mathbb{R}^{d_{\text{seq}}}. \quad (1)$$

We denote the stacked sequence of post representations by $X_{\text{seq}} = [X_1, X_2, \dots, X_T]$. Similarly, given a multi-hot action vector for M actions $a_t \in \{0, 1\}^M$, we form an action representation $A_t = a_t W_a + b_a \in \mathbb{R}^{d_{\text{seq}}}$ using a learnable projection. The stacked action sequence is $A_{\text{seq}} = [A_1, \dots, A_T]$. Finally, the $2T$ -length transformer input is constructed by interleaving post representations and action representations:

$$X_{\text{in}} = [X_1, A_1, X_2, A_2, \dots, X_T, A_T]. \quad (2)$$

In our experiments, we set the transformer embedding dimension to be $d_{\text{model}} = d_{\text{seq}}$.

4.2.2 Transformer architecture and variations. We use a decoder-only transformer with a pre-LayerNorm (Pre-LN) formulation [28], rotary positional embeddings (RoPE) [21], and scaled residual connections. The forward pass of the block is defined in Eqs. (3)–(7).

$$Q, K, V = W_q \text{LN}(X_{\text{in}}), W_k \text{LN}(X_{\text{in}}), W_v \text{LN}(X_{\text{in}}) \quad (3)$$

$$Q_r, K_r = \text{RoPE}(Q, K) \quad (4)$$

$$\text{Attn} = W_o \text{Concat}(\text{SDPA}(Q_r, K_r, V; \text{causal})) \quad (5)$$

$$Y = \text{RescaleAndAdd}(X_{\text{in}}, \text{Attn}) \quad (6)$$

$$Z = \text{RescaleAndAdd}(Y, \text{FFN}(\text{LN}(Y))) \quad (7)$$

We adopt a Pre-LN formulation where $\text{LN}(\cdot)$ denotes Layer Normalization [1] applied to the input of each sublayer. Equation (3) computes query, key, and value projections from the normalized input. Equation (4) applies RoPE to (Q, K) to produce (Q_r, K_r) . Equation (5) performs causal scaled dot-product attention (SDPA). We concatenate attention heads and apply the output projection W_o . Equations (6)–(7) apply scaled residual updates similar to [2]. We

define

$$\text{RescaleAndAdd}(u, v) \triangleq u + \alpha v, \quad (8)$$

where α is a learnable scalar. Finally, the transformer output $Z \in \mathbb{R}^{B \times 2T \times d_{\text{model}}}$ is reduced to $\mathbb{R}^{B \times T \times d_{\text{model}}}$ by discarding odd output positions which correspond to action inputs similar to [30]. The result is passed to the head architecture together with the context features X_{context} for late-fusion prediction; see Sections 4.2.3 and 4.2.4 for details.

The pre-LayerNorm formulation is essential for training stability; without it, we observe training AUC collapsing to 0.5. We also evaluated different attention activations in Eq. (5), including Softmax, Sigmoid, SiLU, and ReLU. Contrary to observations in [30], Softmax matched or exceeded the performance of Sigmoid, SiLU, and ReLU in our setting. Finally, we experimented with several residual/skip-connection variants, including vanilla residual addition, LayerScale [23], scalar rescaling [2], and dense gating [5]. Vanilla residual addition led to intermittent training instability (AUC dropping to 0.5 mid-training). Dense gating, LayerScale, and scalar rescaling all performed within marginal AUC differences. Based on this trade-off, *Feed SR* uses the scalar *RescaleAndAdd* skip connection, which provides stable training with minimal additional complexity.

We also evaluated replacing the *Feed SR* transformer blocks with HSTU layers [30], but observed a consistent performance degradation. For instance, at matched compute (10^{17} FLOPs), the Long Dwell AUC decreases by 0.21% with HSTU. Additional compute-matched comparisons between *Feed SR* and HSTU are provided in Appendix C.

4.2.3 Late Fusion of Features. As described in Section 4.2.1, an interleaved sequence of post representations X_{seq} and action representations A_{seq} are fed to a transformer to obtain embeddings Z . A natural approach is to include all available features in X_{seq} , which we refer to as *early fusion*. We instead use *late fusion* in which context features are concatenated to the transformer output. This is motivated by both efficiency and modeling considerations. First, early fusion is costly. As the feature set grows, early fusion increases the per-token dimension d_{seq} and, consequently, the transformer hidden size d_{model} . This increases parameter count and the compute of both attention and MLP blocks, making it harder to meet production latency and throughput constraints. Second, not all signals add value when modeled as part of the historical sequence. For example, the ablations in Section 4.3 show that candidate popularity is a strong predictor of relevance, but the popularity of posts in a member's history may only be weakly related to the relevance of the current candidate. So, injecting them into every history token can add noise and dilute capacity from more informative behavioral signals. Motivated by these considerations, we restrict the sequential encoder to a compact set of history features that benefit from temporal modeling, and incorporate additional candidate and context features after the transformer.

Offline experiments show only a small degradation (0.07%) in Long Dwell AUC when moving one-third of the features out of the sequence pathway and into late-fusion. This reduction in the sequence feature dimension decreases the transformer dimension and yields an approximately 12% reduction in per-step training time. The features moved to late-fusion are primarily numeric signals that capture item popularity and viewer–author affinity.

Late fusion also simplifies online serving. History item features no longer need to include the context features, since these features are fetched only for the candidate item at inference time. This reduces feature-fetching overhead, decreases the history storage cost, and can lower serving latency. To validate that this architectural change does not degrade the member experience, we conducted an online A/B test comparing late-fusion against early-fusion. The late-fusion model met or exceeded the early-fusion baseline on true north Feed metrics.

4.2.4 Head Architecture. In Section 4.2.3 we introduced the *Feed SR* head architecture, which generates prediction scores. We experiment with several head architectures: (i) **Linear** (a single affine layer), (ii) **MLP** (a two-layer MLP with nonlinearity), (iii) **DCNv2** [26], and (iv) **MMoE** [16]. Table 1 summarizes their relative offline improvements over the Linear baseline.

Head Arch.	Long Dwell AUC Lift	Contributions AUC Lift
Linear	-	-
MLP	+0.30%	+0.54%
DCNv2	+0.32%	+0.54%
MMoE	+0.44%	+0.58%

Table 1: Relative performance improvement of different classification head architectures over the Linear baseline.

We observe that MMoE achieves the best performance among the evaluated heads on both Long Dwell and Contributions responses. In MMoE, the fused representation of transformer output and context features is transformed by a set of shared experts, and each task uses a learned gate to combine expert outputs. A common failure mode is expert collapse, where a small subset of experts dominates across tasks. To mitigate collapse, we apply dropout to the post-softmax gates during training. Following the existing Feed ranking model discussed in Section 3.2, we additionally group tasks into two sets (active vs. passive) for gate routing, which improves performance via shared group-level learning.

4.3 Features

Section 3.2 summarizes the production LinkedIn Feed ranker feature set. *Feed SR* uses a substantially reduced feature set (about 20% of production features), simplifying feature engineering/serving while relying on the transformer to learn many interaction patterns that were previously captured by hand-crafted history transforms.

Motivated by the late-fusion design in Section 4.2.3, we split features into *sequence* features X_{seq} and *context* features X_{context} . X_{seq} focuses on stable identity/semantic signals (e.g., actor/root-actor ID embeddings, content embedding, lightweight categorical/numeric attributes), while X_{context} captures candidate-conditional numeric signals (e.g., popularity, dwell-time buckets, post age, and explicit affinity between viewer and action/root-actor). Additional details on features are provided in Appendix A. Here, *root actor* denotes the creator of the Feed post, and *actor* denotes the member/entity responsible for the resurfaced Feed post, e.g., a reshare, like, etc.

Our ablations highlight three takeaways: (1) despite using far fewer features, *Feed SR* outperforms the production ranker, suggesting the transformer can replace many manually engineered history

transforms; (2) candidate popularity and bucketed dwell-time popularity are still very important for action prediction (e.g., +2.5% Long Dwell AUC), motivating their inclusion as context features fused after the transformer; and (3) while actor/root-actor ID embeddings capture recurring viewer to actor/root actor interaction patterns, explicit viewer to actor/root-actor affinity features remain complementary (e.g., removing viewer-actor affinity yields a 0.3% Long Dwell AUC drop).

We also explored learned root-object/activity ID embeddings, but excluded them due to freshness challenges under once-per-day incremental training and due to table size/latency overhead at the scale of hundreds of millions of item IDs; instead, we rely on nearline-updated popularity and affinity features to capture item-level dynamics at lower operational cost (Appendix A).

4.4 RoPE vs Learned absolute embeddings.

As mentioned in Section 4.2.2, we use rotary positional embeddings (RoPE) [21] to encode token positions. Prior to RoPE, we experimented with additive learned absolute position embeddings, where each token at absolute sequence index t is assigned a learned vector p_t and the transformer input is formed as $x_t + p_t$. In our setting, learned absolute embeddings resulted in unstable average prediction scores during training. We describe this issue in more detail in Appendix D. To mitigate this issue, we replace learned absolute embeddings with RoPE, which encodes position through a shared, deterministic rotation applied to (Q, K) , generating (Q_r, K_r) , rather than a learned lookup table over absolute indices. The rotation is done equally for a Feed item token and its corresponding action token, assigning them the same position in the sequence. We find that RoPE improves score stability, keeping a coefficient of variation for average predicted scores around 1%, and also yields metric gains (e.g., +0.20% Long Dwell AUC) over learned absolute position embeddings.

4.5 Member Profile Embeddings

Member profile embeddings are an LLM-based dense representation that captures comprehensive information from LinkedIn member profiles. These embeddings are generated by aggregating member profile information with a Qwen3 0.6 billion parameter [29] fine-tuned model. To efficiently incorporate this dense embedding into the model, we integrate it as a late-fused context feature, avoiding any increase in the transformer’s dimensionality. Adding profile embeddings is particularly valuable for members with short or sparse histories, where *Feed SR* lacks historical data. Embeddings are refreshed daily. This means that as members evolve their profiles, the embeddings stay aligned with each member’s current professional interests. Empirically, adding profile embeddings as a late-fused dense feature improves Long-Dwell AUC, with more than +2% AUC gains for members with <10 historical actions.

4.6 Training Techniques

4.6.1 Inverse propensity Weighting and Position Debiasing. Position bias is a known phenomenon in ML recommender system models in which recommended items that are shown higher on the list receive more engagement, regardless of quality. To combat position effects, we utilized a combination of 2 approaches.

First, we applied Inverse Propensity Weighting (IPW), a method in which per-position propensity scores for click (or other actions) are computed from offline data, and then the loss corresponding to each post during model training is weighted by the inverse of the propensity score corresponding to the position at which that post was shown. Second, during model training we learn explicit parameters for Feed position [31]. Specifically, we learn a logit offset for the top-60 feed positions for each label and add this to the final logits. The position of an item during online scoring is unknown, but we found that scoring items with the position set to 5 resulted in a well-calibrated model. Using this, we were able to directly use the objective function weights from the existing production model without further tuning.

4.6.2 Incremental Training. Incremental training is a core component of LinkedIn’s online recommendation system and has proven to be both effective and essential for large-scale social media platforms. LinkedIn Feed’s ranking models are updated daily using newly arrived interaction data, and *Feed SR* follows the same paradigm. During cold-start training, the loss is computed over the full interaction history, as described in Section 4.1. During incremental updates, we compute the loss only on newly observed interactions, while still providing the full historical sequence as input to the model. Although daily data is dominated by highly active members, we have not observed evidence so far that this incremental training strategy degrades model quality for less frequent members. Early online experiments at low traffic percentages show additional metric gains beyond those reported in Section 7, which excludes incremental training.

4.6.3 Temporal and Positional Loss Weighting. During training, the model learns equally from all items in the history. However, recent interactions may be more relevant to predict a member’s future behavior. To this end, we introduce two complementary exponential loss decay mechanisms. Position weighting down-weights earlier positions within each training sequence using exponential decay. With half-life set to the sequence length, the first position receives 50% weight while the final (most recent) position receives full weight. Additionally, timestamp weighting applies sample-level decay based on data recency. Each training sample is weighted using exponential decay from a reference end timestamp, with a default 60-day half-life such that two-month-old samples receive 50% weight. Weights are clamped to $[0.0001, 1.0]$ and batch-normalized to maintain gradient scale. Table 2 illustrates offline AUC improvements for *Feed SR* from position-weighted and timestamp-weighted loss enhancements.

Loss	Long Dwell AUC	Contributions AUC
Position-weighted	+0.05%	+0.04%
Timestamp-weighted	+0.07%	+0.23%

Table 2: Relative performance improvements of temporal and positional loss weighting.

4.7 In-Session Leakage

In *Feed SR* we observe a label leakage happening from past interactions in the same session. Unlike non-sequential rankers, *Feed SR*

constructs sequences from user interaction histories, where consecutive tokens in a sequence often belong to the same user session (in-session interactions occur within a 30-minute window). We find that user feedback signals (e.g., dwell time and clicks) are strongly correlated within a session but much less so across sessions. For example, conditioning on the first item in a session having dwell time $> 15s$ increases the likelihood that subsequent items in the same session also exceed 15s, relative to items outside the session. Empirically, this leakage leads to overfitting. This is because during training the model can learn from the previous interactions in the same session. However, during serving in-session engagement labels are not available in the interaction history. To mitigate in-session leakage when constructing training sequences, we experimented with (1) randomizing the order of items within each session and (2) masking items from the same session during attention. While the second approach is more principled, we found that both methods resolved the observed overfitting issues. Furthermore, (2) resulted in slower training due to the data-dependent mask creation. We therefore use simple randomization within the session in the final model.

4.8 Scaling Laws

We perform experiments to observe scaling behavior in *Feed SR*. Figure 2 demonstrates how Long Dwell AUC scales with log of the training FLOPS. Scaling parameters here are the sequence length, the number of transformer layers, and the dimension of ID-embeddings. The figure shows that for every order of magnitude increase in training FLOPS, the evaluation Long Dwell AUC improves by approximately 0.0093, demonstrating a consistent positive scaling effect. Training FLOPS are an approximate function of the number of dense parameters times the sequence length [4]. We also performed ablations that scale model capacity along a single axis at a time: sequence length, number of transformer layers, and ID embedding dimension. Among these, increasing sequence length exhibits the most consistent scaling behavior across metrics (including Long Dwell AUC and Contributions AUC), suggesting that longer histories more effectively utilize available model capacity. In contrast, scaling depth (more layers) or ID embedding dimension in isolation yields mixed and sometimes non-monotonic gains: while Long Dwell AUC often improves, other metrics such as Contributions AUC can plateau or vary irregularly. Overall, these results suggest that to obtain reliable scaling laws across objectives, capacity should be scaled jointly across multiple dimensions rather than by increasing a single dimension in isolation. Additional details and plots are provided in Appendix B.

5 Alternative Approaches Considered

5.1 LLM-Ranker

Before *Feed SR*, we explored an LLM-Ranker system in which all the features of a candidate post were represented as text and passed into an LLM as part of a prompt. Given an input prompt with a particular structure and ending with a question such as “Will the member click on this post?”, the LLM was fine-tuned to output “Yes” or “No” as the next token. The LLM’s output logit for “Yes” was then extracted and interpreted to be the model’s prediction for

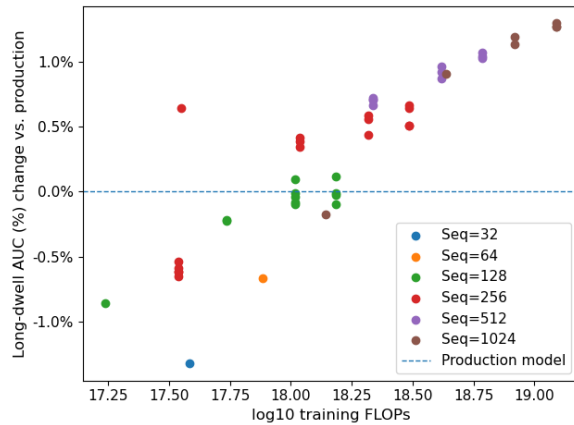


Figure 2: Scaling of Long Dwell AUC as a function of training FLOPs for *Feed SR*. Baseline is the current *Feed* production model.

P(click). The same process was used to make predictions for other probabilities of interest (P(like), P(comment), etc.).

This LLM-Ranker showed promising offline results in early experiments and presented various appealing properties. For instance, because the LLM-Ranker was initialized from an already pre-trained LLM (e.g., LLaMA 3), it already contained a lot of "world knowledge". Furthermore, as the LLM internally also had a transformer backbone, it also benefited from sequential training and multi-item scoring like *Feed SR*.

However, the LLM-Ranker also had several key disadvantages:

- Although certain features were natural to represent as text in the input prompt, it was difficult to encode numeric features as text (e.g., "# of Likes: 382"), and the LLM was not always able to leverage them well to improve the model's prediction performance.
- Because it took hundreds of tokens to represent each post, encoding a member's post interaction history took tens of thousands of tokens, making the model expensive to train and serve due to the large sequence input being passed into the transformer. In contrast, although *Feed SR* also processes a member's interaction history with a transformer, each post in the history is represented only by 2 tokens (item and action embeddings), which is much more efficient.
- The LLM-Ranker never achieved superior online performance over the existing production model. Although it performed well on posts from out-of-network recommendations, the model struggled with network-based recommendations, because it was difficult to encode the strength of network relationships in a text prompt.

Feed SR addresses these limitations through explicit handling of popularity and affinity signals via late fusion, compact post representations that enable efficient serving over long interaction histories, and ID-based embeddings that capture strong network relationships.

5.2 TransAct

We also experimented with augmenting the existing production ranker with history encoding methods such as TransAct [27], Behavioral Sequence Transformer [6], and Deep Interest Network [32]. TransAct improved offline and online metrics. However, it also resulted in big increases in training time and inference latency, especially for longer sequences. While we prototyped the usage of multi-item scoring in TransAct [12], the method was difficult to deploy in a stack implemented for point-wise scoring. Making the sequence the central part of *Feed SR* naturally resolved these challenges by amortizing sequence computation during training and inference.

6 System architecture

6.1 Overview

Figure 3 shows a high level architecture diagram of the inference infrastructure that powers our *Feed SR* system. Our inference system employs a disaggregated architecture that separates CPU and GPU workloads to enable independent scaling and optimal resource utilization. This design consists of two primary components working in concert to deliver efficient, low-latency predictions.

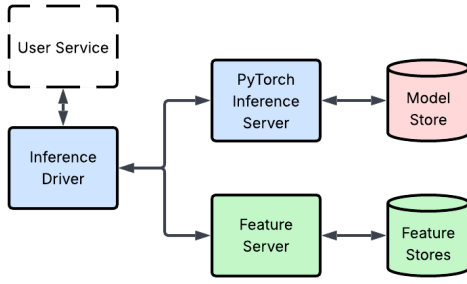
6.1.1 Disaggregated Inference Components. The inference driver is a CPU-based service that handles feature fetching, feature tracking, and request-context-specific CPU-bound transformations. These operations are not GPU-friendly and are therefore isolated from GPU resources. The PyTorch inference server is a Python-based service optimized for GPU execution. It exposes a high-performance gRPC interface that wraps Apache Arrow buffers inside protobuf messages, enabling zero-copy conversion of large data payloads directly to PyTorch tensors. This design eliminates expensive serialization and memory copy operations, allowing the transformer model to process features efficiently on the GPU. The disaggregated setup provides significant operational flexibility, as CPU and GPU workloads can be scaled independently based on their respective bottlenecks and resource requirements.

6.1.2 Efficient Feature Serving. Our feature serving infrastructure minimizes inference latency through precomputation and efficient data layouts. Member history features are generated offline and stored as compact Arrow columnar buffers in key-value stores, enabling efficient access with low memory overhead. Document features are handled similarly and fetched at serving time.

During inference, the driver retrieves and joins member and document features, then transmits the combined feature set as Arrow bytes to the PyTorch inference server. The system relies on zero-copy conversion between Arrow buffers and PyTorch tensors, which is especially important for large history features. Both services are implemented in Python but restricted to high-performance libraries (NumPy, PyTorch, Arrow) to minimize overhead and maximize throughput.

6.2 Improving latency and QPS

6.2.1 CPU-Side Optimization. Our inference pipeline was significantly optimized from an initial loop-based, non-vectorized implementation to eliminate CPU-bound data processing bottlenecks.

Figure 3: System Architecture of *Feed SR*

Two key transformations delivered major speedups: member history parsing improved from 450 ms to 2 ms (225×), and sparse-to-dense tensor conversion from 254 ms to 5 ms per feature (50×). As these operations run thousands of times per second, the gains translate directly into higher production throughput. Member history parsing was reworked using NumPy strided arrays to enable zero-copy, bulk feature processing. This shifted computation from item-centric loops to feature-centric vectorized access, reducing complexity from $O(N \times F)$ to $O(F)$ where N is the number of items and F the number of features, and improving cache locality. Sparse-to-dense conversion was similarly vectorized by replacing nested Python loops with PyTorch tensor indexing and batched assignments, reducing complexity from $O(N \times M \times D)$ to $O(N + M \times D)$ and eliminating per-document overhead, where D is the number of dimensions and M the average number of values per document. Together, these changes apply core performance engineering principles such as zero-copy data movement, columnar layouts, and SIMD-friendly vectorization. This resulted in substantial hardware-level gains: 66% fewer CPU cycles, 71% fewer instructions, 90% fewer cache misses, and 72% fewer branch mispredictions. These improvements are compared to an un-optimized, loop-based implementation. As a result, CPU-bound stages now scale efficiently under high-throughput, real-time inference workloads.

6.2.2 GPU-Side Optimization.

Shared Context Batching. The *Feed SR* inference pipeline scores N candidates (typically $N = 512$) for each member request, using their historical interaction context. Given that the N candidates share the same history, we append all N candidates and compute the scores in a single forward pass via a custom attention mask, similar to M-Falcon [30]. Specifically, we construct an attention mask with two distinct attention patterns: historical context tokens attend to themselves in causal mode and each candidate token attends to all context tokens and itself. This mask is passed to PyTorch’s SDPA (*scaled_dot_product_attention*), enabling parallel scoring of all candidates while preventing cross-candidate leakage. By eliminating the redundant processing, we achieve 80x speedup on the transformer forward pass, for typical workloads with approximately 500 candidates and history length 1000.

Custom Flash Attention Kernel. While shared context batching dramatically reduces computation, PyTorch’s SDPA falls back to a non-Flash Attention implementation when custom attention masks

are provided, forgoing the efficiency benefits of Flash Attention’s tiled computation and online (streaming) softmax.

To address this, we developed a specialized CUDA kernel (*SRMIS*) that extends Flash Attention to support *Feed SR*’s multi-item scoring pattern. Unlike standard approaches requiring explicit mask tensors, *SRMIS* accepts two scalar parameters—*context_length* and *candidate_length*—and implements the requisite attention masking directly within the Flash Attention computation. The attention pattern is fully determined by these parameters: context tokens ($i \leq L$) attend causally to preceding positions, while candidate tokens ($i > L$) attend to all context positions and themselves only.

This design provides three key benefits: (1) *eliminated mask materialization*, avoiding allocation of $O((L + N)^2)$ mask tensors; (2) *compute skipping*, where tiles falling entirely outside the valid attention region are skipped without any computation rather than computed then masked; and (3) *preserved memory efficiency* through Flash Attention’s tiled computation. The *SRMIS* kernel achieves an average 2× speedup over masked SDPA, with exact gains dependent on context and candidate lengths.

6.3 Model Training Optimizations

To improve the efficiency of training for *Feed SR*, the following key optimizations were implemented. These strategies significantly reduced training time, computational overhead, and resource usage.

Optimization Applied	e2e GPU Hours Reduction
Efficient Metrics Computation Kernel	22%
Optimizer Fusing and Gradient Scaling	15%
Fused Data Loading and Processing	50%
Parallelized Evaluation	16%

Table 3: Training performance relative improvements

6.3.1 Efficient Metrics Computation Kernel. The original multilabel AUC computation was performed sequentially, incurring substantial overhead (~ 66 ms per step) due to masked confusion-matrix operations and frequent `cudaStreamSynchronize` calls. To address this, we implemented a custom fused and bucketized CUDA kernel for training-time AUC computation. By integrating the boolean mask directly into the kernel, we eliminated masking overhead and dynamic memory allocation, reducing metric update time from 66 ms to 0.5 ms per step while preserving metric accuracy (negligible AUC error).

6.3.2 Optimizer Fusing and Gradient Scaling. Optimizer inefficiencies were resolved by switching to a fused Adam implementation with integrated gradient scaling. Enabling the fused optimizer flag and incorporating the gradient scaler directly into the CUDA kernel removed redundant `cudaStreamSynchronize` calls, reducing optimizer step time from 40 ms to 20 ms.

6.3.3 Fused Data Loading and Processing. Previously, the data loading process involved significant latency (~ 300 ms) per step due to input/output bottlenecks. To address the data loading bottleneck, we fused padding, batching, and packing operations into a C++ data loader to minimize Python multiprocessing overhead. These changes collectively reduced training step time by >50%

6.3.4 Parallelized Evaluation. Previously, evaluation was performed in a serialized manner (once after several training steps), while evaluation is taking much less GPU memory compared with training, which causes GPU resource waste. In order to improve GPU efficiency, we save all of the checkpoints during training and run evaluation in parallel for all of the checkpoints at once. In this way, we load the dataset only once and run multiple forward passes in parallel to fully utilize the GPU memory, and reduced the overall runtime by 16%

6.4 Additional models in PyTorch inference

Online, the ranking model is also combined with a creator model and a downstream session model. We enable this scoring functionality by implementing a *Scorer* class that acts as the orchestrator for online scoring. This class runs the forward pass for the three different models and applies the objective function weights to derive the final score. We create a single, shared *Scorer* checkpoint using the pre-trained ranking, downstream, creator models as part of the deployment process. We use a HuggingFace-style config.json to store class references as well as the objective function weights.

6.5 Analysis of energy consumption

We analyze the relative energy consumption of *Feed SR* compared to the previous production model. Results are shown in Table 4. The previous production model was trained on the same GPUs, but served online on CPUs. We can see that while training uses more energy for *Feed SR*, inference uses in fact less. One contributing factor is that *Feed SR* is trained on substantially more data, while the number of inference candidates remains unchanged.

Energy Consumption	<i>Feed SR</i> vs. Existing
Training	3.6x
Inference	0.7x

Table 4: Relative energy consumption of *Feed SR* compared to the previous production model during training and inference.

7 A/B Test Results

Online results for *Feed SR* are shown in Table 5. The table shows relative metric changes compared to the existing production ranking model. Overall, *Feed SR* shows +2.10% increase in time spent. Broken down by member segments, we find that *Feed SR* shows the biggest metric gains among the most active member segments while still being positive for less active members and neutral for new members. Note that the online results exclude incremental training (Section 4.6.2).

	Member Segment				
	Overall	DAU	WAU	MAU	New
Time-spent	+2.10%	+2.38%	+1.84%	+0.82%	Not stat-sig

Table 5: Online results of *Feed SR* compared to the production model for time spent.

8 Deployment Lessons

Feed SR represented a complete rewrite of the LinkedIn Feed ranking stack both from a modeling and from an infrastructure perspective. During the process of this work we had several major insights that impacted the course of the project. We discuss these below.

8.1 Avoiding Negative Sampling in Evaluation

While histories are down-sampled, we avoid negative sampling for evaluation candidates to minimize offline-online AUC discrepancies. We found that down-sampling biases evaluation data and inflates offline AUC. Evaluating on the full candidate set yields offline AUCs that better align with online AUCs and top-line metrics.

8.2 Model debugging via offline/online score discrepancy

A key tool in assessing model correctness online during this work has been to compare offline with online scores on the same items. Specifically, we built a pipeline to score *Feed SR* sessions with our offline pipeline. The scores from this pipeline were compared to the logged online scores. Using this process we discovered a large number of bugs in our online stack, whose elimination had a major impact on online metrics.

8.3 Moving away from Java feature transformations

Many among the issues from Section 8.2 were due to offline/online feature discrepancies. Feature transformation was previously handled by shared Java based transformations. With PyTorch, this shared framework is gone and a new one needs to be built. While current online feature transformation is handled by a mix of Java and NumPy/PyTorch transformation, we are building a more principled approach to create Python based shared feature transformations.

9 Conclusion

In this paper we have presented *Feed SR*, a large-scale sequential recommendation model for LinkedIn's Feed. We have illustrated the modeling choices and system design that allowed us to create a better member experience than the previously existing production ranker. *Feed SR* is now the majority member experience on LinkedIn's Feed.

Acknowledgments

This work represents the joint efforts across multiple teams in LinkedIn without whom this would not have been possible. We would like to thank Deepak Agarwal, Tim Jurka, Balaji Krishnapuram, Xiaobing Xue, Hristo Danchev, Christine Lin, Sudarshan Ramanujam, Shihai He for supporting this work.

References

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. Layer normalization. *arXiv preprint arXiv:1607.06450* (2016).
- [2] Thomas Bachlechner, Bodhisattwa Prasad Majumder, Henry Mao, Gary Cottrell, and Julian McAuley. 2021. Rezero is all you need: Fast convergence at large depth. In *Uncertainty in Artificial Intelligence*. PMLR, 1352–1361.

- [3] Fedor Borisjuk, Mingzhou Zhou, Qingquan Song, Siyu Zhu, Birjodh Tiwana, Ganesh Parameswaran, Siddharth Dangi, Lars Hertel, Qiang Charles Xiao, Xiaochen Hou, et al. 2024. LiRank: Industrial Large Scale Ranking Models at LinkedIn. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4804–4815.
- [4] Adam Casson. 2023. Transformer FLOPs. <https://adamcasson.com/posts/transformer-flops>. Accessed: 2026-01-28.
- [5] Yekun Chai, Shuo Jin, and Xinwen Hou. 2020. Highway transformer: Self-gating enhanced self-attentive networks. *arXiv preprint arXiv:2004.08178* (2020).
- [6] Qiwei Chen, Huan Zhao, Wei Li, Pipei Huang, and Wenwu Ou. 2019. Behavior sequence transformer for e-commerce recommendation in alibaba. In *Proceedings of the 1st international workshop on deep learning practice for high-dimensional sparse data*. 1–4.
- [7] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ipsir, et al. 2016. Wide & Deep Learning for Recommender Systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems (DLRS)*.
- [8] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2205.14135>
- [9] Jaixin Deng, Shiyao Wang, Kuo Cai Li, Qigen Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment. *arXiv preprint arXiv:2502.18596* (2025). <https://arxiv.org/abs/2502.18596>
- [10] Swapnil Ghike and Shubham Gupta. 2016. FollowFeed: LinkedIn’s feed made faster and smarter. <https://www.linkedin.com/blog/engineering/feed/followfeed-linked-in-s-feed-made-faster-and-smarter>
- [11] Ruidong Han, Bin Yin, Shangyu Chen, He Jiang, Fei Jiang, Xiang Li, Chi Ma, Mincong Huang, Xiaoguang Li, Chunzhen Jing, Yueming Han, Menglei Zhou, Lei Yu, Chuan Liu, and Wei Lin. 2025. MTGR: Industrial-Scale Generative Recommendation Framework in Meituan. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM)*. <https://arxiv.org/abs/2505.18654>
- [12] Lars Hertel, Neil Daftary, Fedor Borisjuk, Aman Gupta, and Rahul Mazumder. 2024. Efficient user history modeling with amortized inference for deep learning recommendation models. *arXiv preprint arXiv:2412.06924* (2024).
- [13] Dmitry Ivanov, Dennis Van Der Staay Ichen, Taylor Colin, Liu King, Fen Will, Kindi Rahal, Anirudh Sudarshan, and Shahin Safai. 2022. TorchRec: A PyTorch Domain Library for Recommendation Systems. In *Proceedings of the 16th ACM Conference on Recommender Systems (RecSys)*.
- [14] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. In *Proceedings of the IEEE International Conference on Data Mining (ICDM)*. <https://arxiv.org/abs/1808.09781>
- [15] Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H. Chi. 2018. Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*.
- [16] Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H Chi. 2018. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1930–1939.
- [17] Maxim Naumov, Deepak Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, et al. 2019. Deep Learning Recommendation Model for Personalization and Recommendation Systems. *arXiv preprint arXiv:1906.00091* (2019). <https://arxiv.org/abs/1906.00091>
- [18] Qiji Pei, Changhua Lv, Chao Li, Junfeng Ge, and Wenwu Ou. 2021. End-to-end User Behavior Retrieval in Click-through Rate Prediction Model. In *Proceedings of the 28th International Conference on Machine Learning (ICML)*.
- [19] Sudarshan Srinivasa Ramanujam, Antonio Alonso, Saurabh Kataria, Siddharth Dangi, Akhilesh Gupta, Birjodh Singh Tiwana, Manas Somaiya, Luke Simon, David Byrne, Sojeong Ha, et al. 2025. Large Scale Retrieval for the LinkedIn Feed using Causal Language Models. *arXiv preprint arXiv:2510.14223* (2025).
- [20] Sudarshan Srinivasa Ramanujam, Akanksha Bindal, Yu Jiang, Timothy J Hazen, David Golland, Fengyu Zhang, Daqi Sun, Wanning Li, Birjodh Singh Tiwana, Siddharth Dangi, et al. 2025. LinkedIn Post Embeddings: Industrial Scale Embedding Generation and Usage across LinkedIn. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 6038–6044.
- [21] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing* 568 (2024), 127063.
- [22] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*. <https://arxiv.org/abs/1904.06690>
- [23] Hugo Touvron, Matthieu Cord, Alexandre Sablayrolles, Gabriel Synnaeve, and Hervé Jégou. 2021. Going deeper with image transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*. 32–42.
- [24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [25] Ruoxi Wang, Bin Fu, Gang Fu, and Ming Wang. 2021. DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems. In *Proceedings of the Web Conference (WWW)*.
- [26] Ruoxi Wang, Rakesh Shivanna, Derek Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed Chi. 2021. Dcn v2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. In *Proceedings of the web conference 2021*. 1785–1797.
- [27] Xue Xia, Pong Eksombatchai, Nikil Pancha, Dhruvil Deven Badani, Po-Wei Wang, Neng Gu, Saurabh Vishwas Joshi, Nazanin Farahpour, Zhiyuan Zhang, and Andrew Zhai. 2023. Transact: Transformer-based realtime user action model for recommendation at pinterest. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5249–5259.
- [28] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tieyan Liu. 2020. On layer normalization in the transformer architecture. In *International conference on machine learning*. PMLR, 10524–10533.
- [29] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [30] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152* (2024).
- [31] Zhe Zhao, Lichan Hong, Li Wei, Jilin Chen, Aniruddh Nath, Shawn Andrews, Aditee Kumthekar, Maheswaran Sathiamoorthy, Xinyang Yi, and Ed Chi. 2019. Recommending what video to watch next: a multitask ranking system. In *Proceedings of the 13th ACM conference on recommender systems*. 43–51.
- [32] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1059–1068.

A Feature definitions and full list

Terminology. We use *root actor* to denote the creator of the original feed post, and *actor* to denote the member/entity responsible for the surfaced feed unit (e.g., a reshare) or the entity that performed an activity (e.g., like). The original post is referred to as the *root object*, and subsequent events such as reshares are referred to as *activities*. *Affinity* denotes the connection strength between a viewer and an actor/root actor, measured via historical engagement signals.

Sequence features (X_{seq}). Sequence features are defined per item in the interaction history (one feature vector per sequence position) and form the per-timestep input to the transformer. We include:

- **Actor/root-actor hashed ID embeddings.** Learned hashed ID embeddings for actor ID and root actor ID using a shared embedding table. Engagements on LinkedIn feed are driven by viewer's network where viewers tend to have repeated interactions within their network and have recurring preferences towards specific actors/root actors. Actor/root actor ID embedding is an essential to capture this viewer to actor/root actor affinity signal.
- **Content embedding.** A 50-dimensional post embedding from LinkedIn's post embedding model [20].
- **Categorical features.** Actor type and root actor type (e.g., member, company), verb type (e.g., share, repost), object type (e.g., article, multi-photo, video), viewer device OS, and whether the viewer is connected to the root actor.
- **Numeric features.** Actor popularity/engagement aggregated across time windows; viewer-actor dwell-time affinity across multiple windows/quantiles; and viewer network size.

Context features ($X_{context}$). Context features describe the candidate post and the viewer-candidate relationship at prediction time, and are concatenated with the transformer output in the late-fusion head. These features are primarily numeric and include:

- **Viewer-(root-)actor affinity.** Time-segmented vectors capturing clicks/likes/impressions over multiple horizons (e.g., 7–365 days).
- **Candidate popularity.** Candidate-level popularity counts for the root object and activity (e.g., clicks/likes/impressions).
- **Bucketed dwell-time popularity.** Bucketed dwell-time counts for the candidate (e.g., 0–5s up to >60s), which is particularly important for Long Dwell prediction (e.g., +2.5% absolute Long Dwell AUC lift in our ablations).
- **Additional context.** Feed post age and viewer network strength.

Why we do not use root-object/activity ID embedding tables. We explored learned ID embeddings for root object ID and activity ID, but excluded them due to:

- (1) **Freshness and non-stationarity.** Feed activities and popularity evolve rapidly and benefit from nearline updates; once-per-day incremental training is insufficient to keep large ID embedding tables fresh without continual-learning infrastructure.

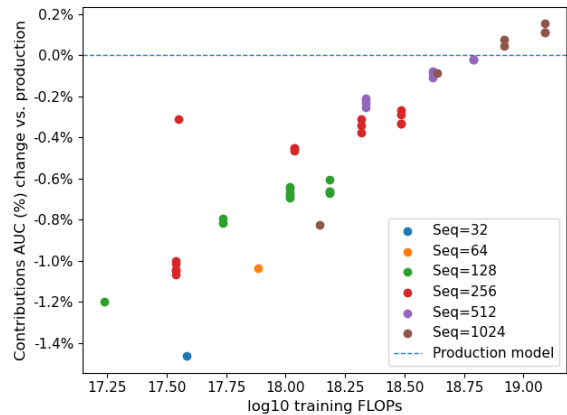


Figure 4: Scaling of Contributions AUC as a function of training FLOPs for *Feed SR*.

- (2) **Scale and latency.** Object IDs can number in the hundreds of millions, leading to very large embedding tables that increase training cost and serving latency/memory footprint. Instead, we rely on nearline-updated popularity and affinity features (often aggregated over long horizons) to retain most of the predictive benefit with lower operational cost.

B Scaling Laws

In Section 4.8 we illustrated scaling laws for *Feed SR*. Here, we expand on those results. Figure 4 provides Contributions AUC scaling with log of the training FLOPs.

Furthermore, we discuss results of scaling sequence length, number of transformer layers and ID embedding dimension individually. Figures 6, 7, and 8 show Long Dwell AUC and contribution AUC when scaling sequence length, number of transformer layers, or ID embedding dimension while holding everything else constant. From these plots we make the following observations about scaling laws when model capacity is scaled in isolation. Increasing the sequence length consistently demonstrates clear scaling laws across all evaluation metrics, including Long Dwell AUC, and Contributions AUC. This suggests that longer sequences effectively leverage the model's capacity for improved performance. Similar observation can be made from Figure 8. In contrast, independently increasing the ID embedding dimension or the number of transformer layers does not consistently demonstrate scaling laws for Contribution AUC. While some metrics, such as Long Dwell AUC, show an upward trend with scaling, others, like Contributions AUC, exhibit irregular or plateauing behavior. These observations suggest that to fully realize scaling laws across all metrics, it is necessary to scale multiple dimensions simultaneously rather than in isolation.

C Scaling laws: *Feed SR* vs. HSTU

We extend the comparison between *Feed SR* and HSTU [30] discussed in Section 4.8. Figures 9 and 10 report compute-matched scaling results for Long Dwell and Contributions AUC. Across the range of compute we were able to evaluate, HSTU underperforms *Feed SR* on both metrics. We want to state that the evaluation results

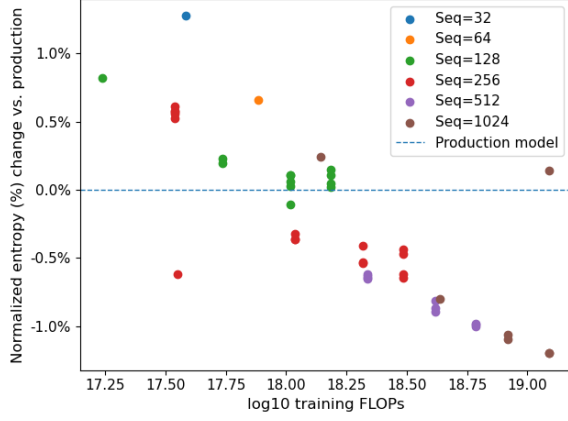


Figure 5: Scaling of normalized evaluation entropy as a function of training FLOPs for *Feed SR*.

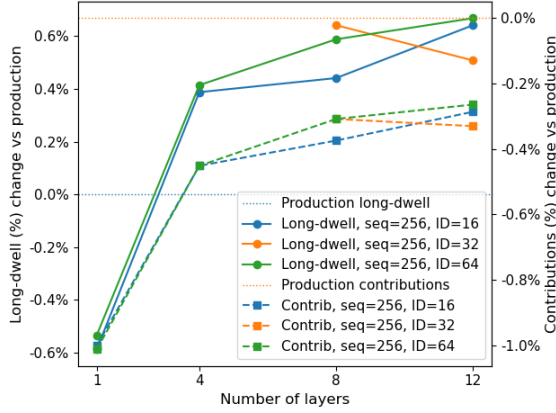


Figure 6: Scaling number of transformer layers while keeping all other hyper parameters (sequence length, id embedding dimension) constant.

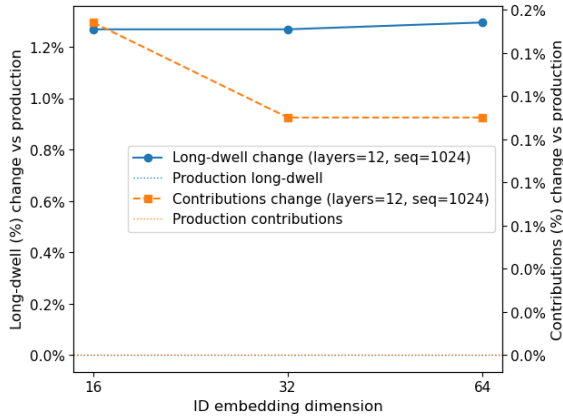


Figure 7: Scaling id embedding dimension while keeping all other hyper parameters (number of transformer layers, sequence length) constant.

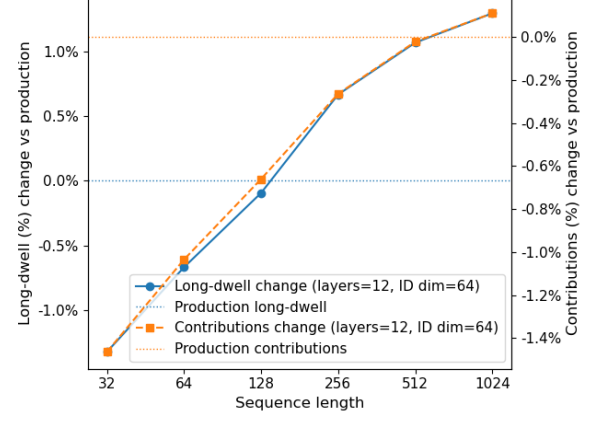


Figure 8: Scaling sequence length while keeping all other hyper parameters (number of transformer layers, id embedding dimension) constant.

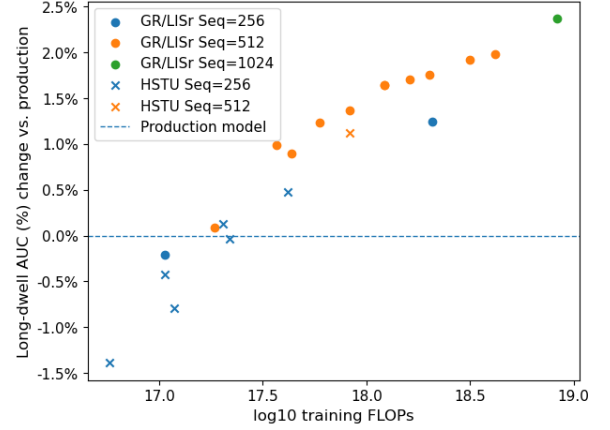


Figure 9: Scaling of Long Dwell AUC as a function of training FLOPs for *Feed SR* vs *HSTU*.

in Figure 9 and 10 were obtained on a different evaluation dataset than the results in Figure 2, 4, 5, 6, 7, 8.

We train HSTU models using the open-source implementation released by the authors¹. For larger model configurations, the open-source HSTU code runs out of memory; therefore, we omit HSTU results beyond $\log(\text{FLOPs}) = 18$. In contrast, we are able to train larger configurations with the *Feed SR* transformer blocks using open-source tooling, in part because they are compatible with standard FlashAttention [8], whereas the HSTU implementation is not.

D Score Instability with Position Embeddings

We hypothesize that this instability is driven by data imbalance across absolute positions. The position-embedding table contains parameters for all indices up to $T_{\max}$, but the empirical distribution of sequence lengths is highly skewed (Figure 12). Most training examples have short histories, so embeddings corresponding to large

¹<https://github.com/meta-recsys/generative-recommenders>

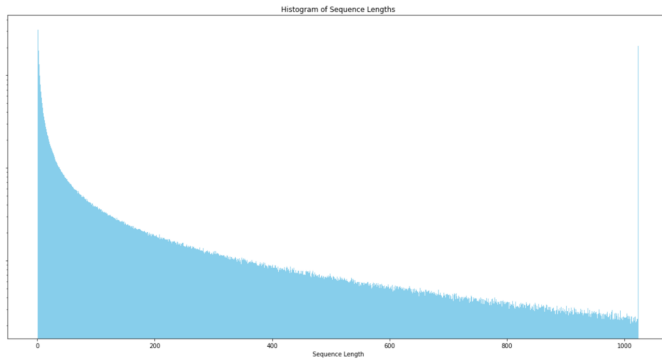


Figure 12: Histogram of viewer sequence lengths in the training dataset. The distribution is highly skewed, with a long tail of short histories; as a result, large absolute position indices are under-represented during training. Y-axis values are omitted due to legal reasons.

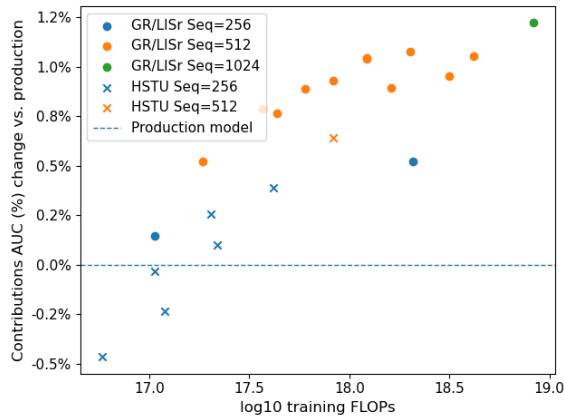


Figure 10: Scaling of Contributions AUC as a function of training FLOPs for *Feed SR* and *HSTU*.

absolute indices receive few updates and are under-trained. This is especially harmful for highly active users with long interaction histories, where tokens at large absolute positions are common and contribute meaningfully to prediction quality.

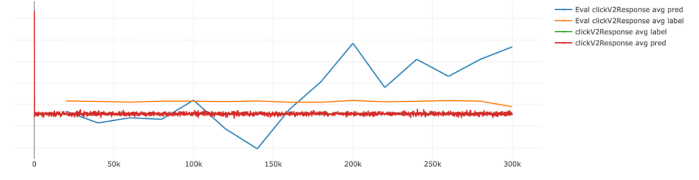


Figure 11: Average label and average prediction score versus training steps when using learned absolute position embeddings. The average label (orange) remains stable, while the average prediction (blue) exhibits large drift, indicating poor calibration. Y-axis values are omitted due to legal reasons.

E Longer Viewer Interaction History Period

We experimented with using 6 months versus 1 year of historical data to construct viewer history sequences, while keeping the maximum sequence length fixed at 1000, as described in Section 4.1. Using 1 year of history resulted in a 0.21% Long Dwell AUC lift and a 0.15% Contribution AUC lift. This change also led to a 12% increase in training time due to the longer history of members. The new sequence also increases member coverage by +20%.

Received 8 February 2026